TRƯỜNG ĐẠI HỌC CỬU LONG · KHOA CNTT – TRÍ TUỆ NHÂN TẠO

BÀI GIẢNG KIỂM THỬ PHẦN MỀM

CHƯƠNG 1

TỔNG QUAN VỀ KIỂM THỬ PHẦN MỀM

Khái niệm – Lỗi & chất lượng – Nguyên tắc – Quy trình – Con người trong kiểm thử

Thời lượng: **4 tiết** (1 tiết = 45 phút)

Biên soạn: ThS. Lôi Thị Tú Trân

Vĩnh Long – 2026

Mục tiêu chương học

Kiến thức

Trình bày được khái niệm kiểm thử phần mềm, mục tiêu kiểm thử và phân biệt được ba khái niệm Error – Defect – Failure.

Kiến thức

Giải thích được 7 nguyên tắc kiểm thử (testing principles) theo ISTQB và các cách tiếp cận kiểm thử.

Kỹ năng

Mô tả và vận dụng được quy trình kiểm thử phần mềm vào một tình huống dự án cụ thể.

Kỹ năng

Phân tích được một tình huống lỗi thực tế: chỉ ra nguyên nhân gốc (root cause) và đề xuất cách khắc phục.

Thái độ

Hình thành tư duy phản biện (critical thinking), khách quan – đặc trưng của người kiểm thử (tester).

Thái độ

Nhận thức chất lượng là trách nhiệm của cả nhóm (whole-team approach), không riêng tester.

Chuẩn tham chiếu: **ISTQB CTFL v4.0** – Chapter 1: Fundamentals of Testing.

Nội dung chương 1

- **1.1** Khái niệm và mục tiêu của kiểm thử phần mềm (Software Testing)
- **1.2** Kiểm thử và gỡ lỗi (Testing vs. Debugging)
- **1.3** Lỗi và chất lượng phần mềm: Error – Defect – Failure
- **1.4** Sự cần thiết của kiểm thử phần mềm
- **1.5** Bảy nguyên tắc kiểm thử (Seven Testing Principles)
- **1.6** Các cách tiếp cận trong kiểm thử (Testing Approaches)
- **1.7** Quy trình kiểm thử phần mềm (Test Process)
- **1.8** Con người trong kiểm thử: vai trò, tâm lý, kỹ năng
- **1.9** Bài tập vận dụng + Câu hỏi ôn tập

THỜI LƯỢNG: ≈ 0,5 TIẾT

1.1

Khái niệm và mục tiêu của kiểm thử phần mềm

- Định nghĩa kiểm thử
- Hiểu lầm thường gặp
- Mục tiêu kiểm thử theo ISTQB

Kiểm thử phần mềm là gì?

Kiểm thử phần mềm (Software Testing) là quá trình **đánh giá (evaluation)** và **xác minh (verification)** rằng một phần mềm đáp ứng các yêu cầu đã được đặc tả và hoạt động đúng như mong đợi.

GÓC NHÌN MỞ RỘNG THEO ISTQB

Không chỉ là "chạy thử"

Kiểm thử bao gồm cả các hoạt động **trước** và **sau** khi thực thi: lập kế hoạch, phân tích, thiết kế, báo cáo, kết thúc.

Gồm cả tĩnh và động

Kiểm thử **tĩnh (static)**: rà soát tài liệu, mã nguồn – không chạy chương trình.

Kiểm thử **động (dynamic)**: thực thi phần mềm với dữ liệu đầu vào.

Là hoạt động cả vòng đời

Kiểm thử diễn ra **xuyên suốt SDLC**, không phải một giai đoạn nằm cuối dự án.

Ghi nhớ: Kiểm thử ≠ chỉ tìm lỗi. Kiểm thử còn nhằm **xây dựng niềm tin** vào chất lượng và **cung cấp thông tin** cho việc ra quyết định.

Mục tiêu của kiểm thử (Test Objectives – ISTQB)

1. Ngăn ngừa khiếm khuyết

Đánh giá sản phẩm công việc (yêu cầu, thiết kế) ngay từ sớm để **phòng ngừa (prevent)** lỗi phát sinh trong mã nguồn.

2. Xác minh yêu cầu

Kiểm tra tất cả các yêu cầu đã được **cài đặt đầy đủ (verification)**.

3. Kiểm tra sự phù hợp

Xác nhận đối tượng kiểm thử hoạt động đúng **kỳ vọng người dùng (validation)**.

4. Xây dựng niềm tin

Tạo **sự tự tin (confidence)** về mức chất lượng của đối tượng kiểm thử.

5. Tìm khiếm khuyết & thất bại

Phát hiện **defect / failure** càng sớm càng tốt để giảm rủi ro.

6. Cung cấp thông tin

Cung cấp đủ thông tin cho **các bên liên quan (stakeholders)** ra quyết định phát hành.

7. Giảm rủi ro

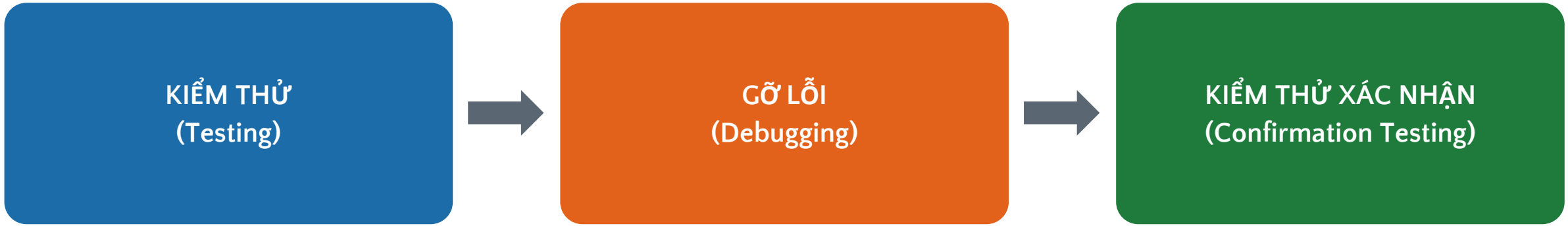
Giảm mức **rủi ro chất lượng (quality risk)** còn tồn đọng của sản phẩm.

8. Tuân thủ

Đáp ứng các yêu cầu **hợp đồng, pháp lý, tiêu chuẩn ngành** (banking, y tế, hàng không...).

Mục tiêu kiểm thử **thay đổi theo ngữ cảnh**: kiểm thử thành phần chú trọng tìm lỗi sớm; kiểm thử chấp nhận chú trọng xây dựng niềm tin.

Kiểm thử và Gỡ lỗi (Testing vs. Debugging)



Phát hiện **thất bại (failure)** do khiếm khuyết gây ra

Tìm **nguyên nhân**, phân tích và **sửa** khiếm khuyết

Chạy lại để **xác nhận** lỗi đã được sửa đúng

Ai làm gì? Kiểm thử thường do **tester** thực hiện; gỡ lỗi do **lập trình viên (developer)** thực hiện.
Trong Agile/DevOps, ranh giới này mờ đi: tester có thể tham gia gỡ lỗi, developer viết và chạy unit test.

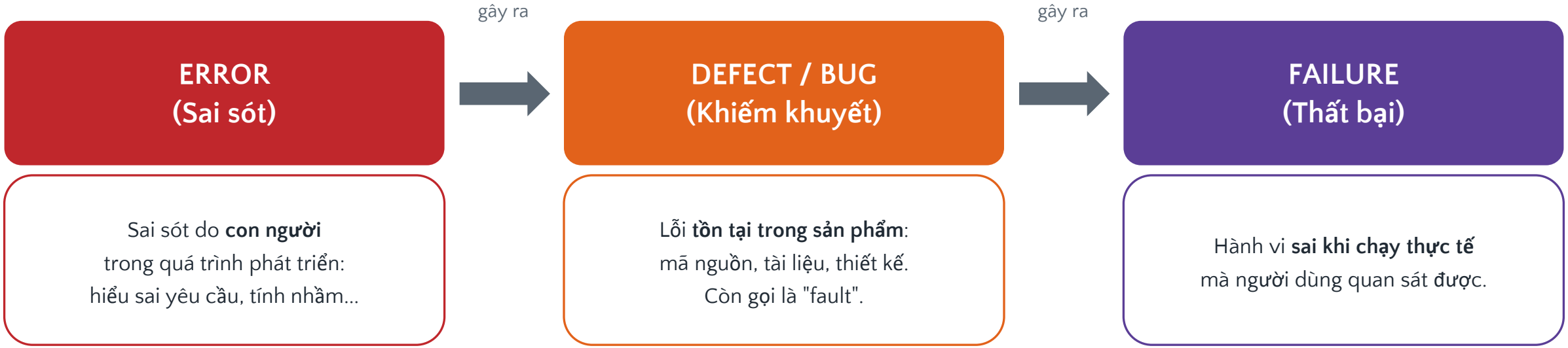
THỜI LƯỢNG: ≈ 0,75 TIẾT

1.3

Lỗi và chất lượng phần mềm

- Error – Defect – Failure
- Nguyên nhân gốc (Root cause)
- Chi phí sửa lỗi
- Chất lượng phần mềm

Error – Defect – Failure: chuỗi nhân quả



Lưu ý quan trọng (ISTQB):

- Không phải mọi khiếm khuyết đều gây thất bại – có defect **nằm im** vì đoạn mã đó không bao giờ được thực thi.
- Ngược lại, một thất bại có thể do **điều kiện môi trường** (nhiễu điện từ, lỗi phần cứng, cấu hình sai) chứ không do defect.
- **Root cause (nguyên nhân gốc):** lý do sâu xa sinh ra error – ví dụ tài liệu yêu cầu mơ hồ, thiếu đào tạo, áp lực tiến độ.

Ví dụ minh họa: chuỗi Error → Defect → Failure

Hệ thống bán hàng – chức năng tính chiết khấu (discount).

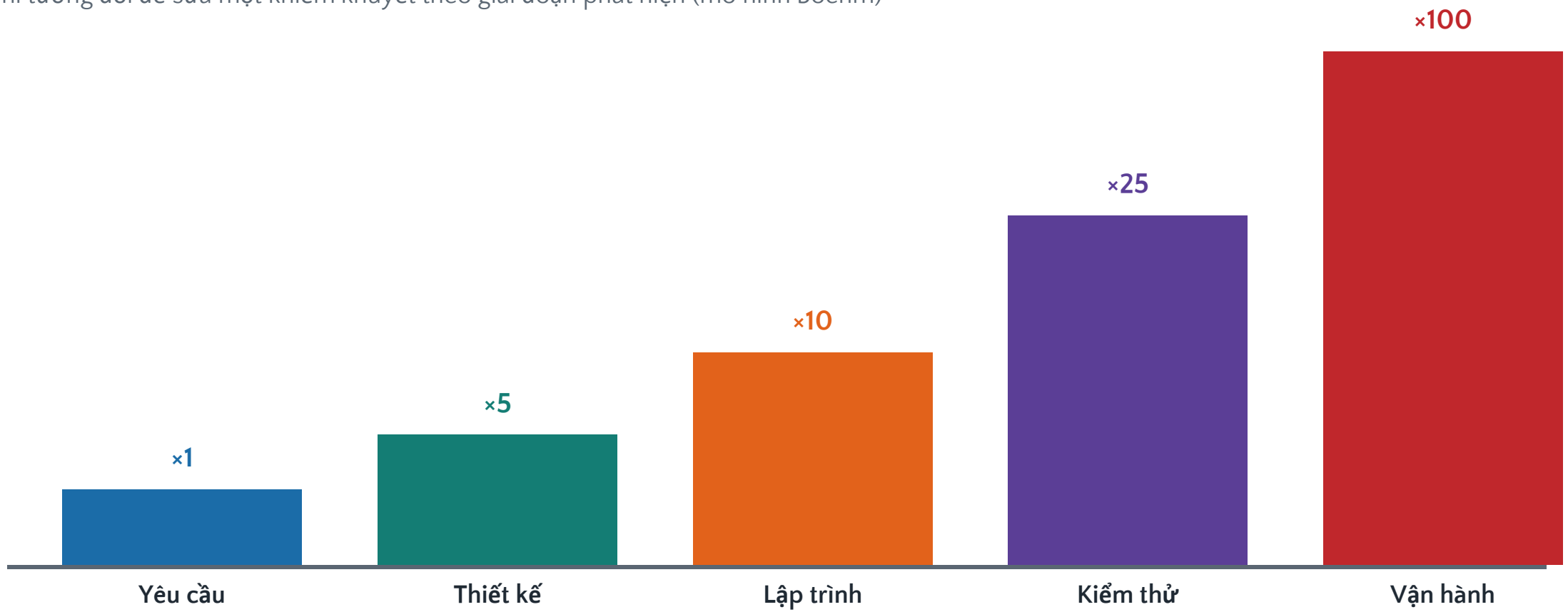
```
// YÊU CẦU (Requirement):
//   "Giảm 10% cho đơn hàng có giá trị LỚN HƠN 1.000.000 đồng"

// MÃ NGUỒN THỰC TẾ (Actual code):
if (total > 500000) {
    discount = total * 0.1;
}
```

Error: lập trình viên hiểu sai ngưỡng (1.000.000 → 500.000) · **Defect:** điều kiện `total > 500000` sai trong code · **Failure:** đơn 600.000đ vẫn được giảm 10% → doanh nghiệp thất thu.

Chi phí sửa lỗi tăng theo giai đoạn phát hiện

Chi phí tương đối để sửa một khiếm khuyết theo giai đoạn phát hiện (mô hình Boehm)



Đây là cơ sở của nguyên tắc **"Kiểm thử sớm" (Early testing / Shift-left)**: phát hiện lỗi ở giai đoạn yêu cầu rẻ hơn hàng chục tới hàng trăm lần so với khi phần mềm đã vận hành.

Chất lượng phần mềm – 8 đặc tính theo ISO/IEC 25010

Tính phù hợp chức năng (Functional suitability)

Phần mềm làm **đúng** những gì được yêu cầu: đầy đủ, chính xác, phù hợp.

Hiệu năng (Performance efficiency)

Thời gian phản hồi, thông lượng, mức sử dụng tài nguyên.

Tính tương thích (Compatibility)

Cùng tồn tại và trao đổi dữ liệu tốt với hệ thống khác.

Khả năng sử dụng (Usability)

Dễ học, dễ dùng, giao diện thân thiện, hỗ trợ người khuyết tật.

Độ tin cậy (Reliability)

Hoạt động ổn định, chịu lỗi, phục hồi được sau sự cố.

Bảo mật (Security)

Bảo vệ dữ liệu: bí mật, toàn vẹn, xác thực, chống chối bỏ.

Khả năng bảo trì (Maintainability)

Dễ đọc, dễ sửa, dễ kiểm thử, module hoá tốt.

Tính khả chuyển (Portability)

Dễ cài đặt, thay thế, chạy trên nhiều môi trường.

Giáo trình tóm gọn thành 5 tiêu chí: **đúng chức năng – ổn định – dễ sử dụng – an toàn – dễ bảo trì**. ISO 25010 là bản chi tiết hoá quốc tế của các tiêu chí này.

Vì sao phải kiểm thử phần mềm?

Phần mềm luôn tiềm ẩn lỗi

Con người viết phần mềm ⇒ con người mắc sai sót. Hệ thống càng lớn, xác suất lỗi càng cao.

Hậu quả có thể nghiêm trọng

Thiệt hại tài chính, mất dữ liệu, tổn hại uy tín, thậm chí **nguy hiểm tính mạng** (y tế, hàng không).

Giảm chi phí sửa lỗi

Phát hiện sớm ⇒ chi phí khắc phục thấp hơn nhiều lần (xem mô hình chi phí ở trên).

Tăng độ tin cậy

Sản phẩm ổn định hơn, ít sự cố khi vận hành thực tế.

Cải thiện trải nghiệm

Người dùng hài lòng ⇒ giữ chân khách hàng, tăng doanh thu.

Bảo vệ uy tín tổ chức

Một sự cố lớn có thể xoá bỏ nhiều năm xây dựng thương hiệu.

Sự cố kinh điển: Ariane 5 (1996) nổ sau 37 giây do lỗi ép kiểu số – thiệt hại ~370 triệu USD; Therac-25 gây tử vong do lỗi phần mềm điều khiển liều xạ trị.

THỜI LƯỢNG: ≈ 0,75 TIẾT

1.5

Bảy nguyên tắc kiểm thử (Seven Testing Principles)

- 7 nguyên tắc nền tảng theo ISTQB
- Ý nghĩa thực tiễn của từng nguyên tắc

Bảy nguyên tắc kiểm thử

1. Kiểm thử chỉ chứng minh sự hiện diện của lỗi

Testing shows the presence, not the absence of defects.

Tìm được lỗi ⇒ chắc chắn có lỗi. Không tìm được lỗi ⇒ **KHÔNG** chứng minh phần mềm không có lỗi.

2. Không thể kiểm thử vét cạn

Exhaustive testing is impossible.

Số tổ hợp đầu vào là **vô hạn trên thực tế** ⇒ phải dùng **phân tích rủi ro** và **kỹ thuật thiết kế test** để chọn tập kiểm thử tiêu biểu.

3. Kiểm thử sớm tiết kiệm chi phí

Early testing saves time and money (**shift-left**).

Bắt đầu kiểm thử tĩnh ngay từ tài liệu yêu cầu, không đợi có mã nguồn.

4. Lỗi thường tập trung theo cụm

Defects cluster together.

Theo quy tắc Pareto, ~80% lỗi nằm trong ~20% mô-đun ⇒ **ưu tiên** kiểm thử vùng rủi ro cao.

5. Nghịch lý thuốc trừ sâu

Pesticide paradox.

Lặp mãi một bộ test ⇒ không còn tìm được lỗi mới. Phải **rà soát và bổ sung** test case liên tục.

6. Kiểm thử phụ thuộc ngữ cảnh

Testing is context dependent.

Kiểm thử app thương mại điện tử khác hoàn toàn kiểm thử phần mềm điều khiển thiết bị y tế.

7. Ngộ nhận "không lỗi"

Absence-of-errors fallacy.

Sửa hết lỗi nhưng phần mềm **không đáp ứng nhu cầu người dùng** thì vẫn là sản phẩm thất bại.

Nguyên tắc kiểm thử → Hành động cụ thể của tester

Nguyên tắc	Tester phải làm gì trong dự án?
1. Chỉ chứng minh có lỗi	Không tuyên bố "phần mềm không còn lỗi"; báo cáo theo **mức độ rủi ro còn lại**.
2. Không kiểm thử vét cạn	Áp dụng **phân vùng tương đương, giá trị biên, bảng quyết định** để giảm số test case.
3. Kiểm thử sớm	Tham gia **review tài liệu yêu cầu / user story** ngay từ sprint planning.
4. Lỗi tập trung	Phân tích **lịch sử lỗi (defect history)** để xác định mô-đun rủi ro cao, tăng độ sâu kiểm thử.
5. Nghịch lý thuốc trừ sâu	Định kỳ **rà soát bộ hồi quy (regression suite)**, bổ sung test mới, dùng kiểm thử thăm dò.
6. Phụ thuộc ngữ cảnh	Điều chỉnh **mức độ chính thức và độ bao phủ** theo rủi ro và lĩnh vực nghiệp vụ.
7. Ngộ nhận không lỗi	Phối hợp với **BA / Product Owner** để kiểm chứng phần mềm đúng nhu cầu thật.

Các cách tiếp cận trong kiểm thử (Testing Approaches)

Dựa trên rủi ro (Risk-based)

Ưu tiên kiểm thử phần có **xác suất lỗi cao** hoặc **tác động lớn**. Rủi ro = Xác suất × Mức thiệt hại.

Dựa trên phân tích (Analytical)

Xây dựng test case **có hệ thống** từ tài liệu yêu cầu, đặc tả, thiết kế.

Dựa trên mô hình (Model-based)

Dùng **sơ đồ trạng thái, luồng nghiệp vụ, mô hình dữ liệu** để sinh test.

Có hệ thống / theo kỹ thuật (Methodical)

Áp dụng bộ kỹ thuật chuẩn: phân vùng tương đương, giá trị biên, checklist chất lượng.

Dựa trên kinh nghiệm (Experience-based)

Dựa vào hiểu biết và trực giác của tester: **đoán lỗi, kiểm thử thăm dò**.

Linh hoạt / phản ứng (Reactive)

Điều chỉnh hoạt động kiểm thử **theo kết quả thu được** trong quá trình chạy test.

Nguyên tắc định hướng tư duy – Cách tiếp cận cung cấp phương pháp triển khai. Dự án thực tế thường **phối hợp nhiều cách tiếp cận**.

THỜI LƯỢNG: ≈ 1 TIẾT

1.7

Quy trình kiểm thử phần mềm (Test Process)

- 4 bước cơ bản
- 7 hoạt động kiểm thử theo ISTQB
- Testware & truy vết

Quy trình kiểm thử – 4 bước cơ bản

BƯỚC 1

Lập kế hoạch kiểm thử (Test Planning)

Xác định mục tiêu, phạm vi, nguồn lực, thời gian, công cụ, rủi ro và **tiêu chí kết thúc (exit criteria)**.

BƯỚC 2

Phân tích & thiết kế kiểm thử (Analysis & Design)

Đọc hiểu yêu cầu, xác định **điều kiện kiểm thử**, thiết kế **test case** và **test data**, chọn kỹ thuật phù hợp.

BƯỚC 3

Triển khai & thực thi (Implementation & Execution)

Chuẩn bị môi trường, thực thi test case, ghi nhận **Pass/Fail**, báo cáo lỗi, **kiểm thử lại (retest)** sau khi sửa.

BƯỚC 4

Đánh giá & kết thúc (Evaluation & Closure)

So sánh với tiêu chí kết thúc, tổng hợp **báo cáo kiểm thử**, lưu trữ testware, **rút kinh nghiệm (lessons learned)**.

Quy trình này có **tính lặp (iterative)**: khi lỗi được sửa hoặc yêu cầu thay đổi, ta quay lại bước thiết kế / thực thi.

Bảy hoạt động kiểm thử theo ISTQB CTFL v4.0

1	**Lập kế hoạch** **Test planning**	Xác định mục tiêu, cách tiếp cận, nguồn lực, lịch trình.
2	**Giám sát & kiểm soát** **Monitoring & control**	So sánh tiến độ thực tế với kế hoạch; đưa ra hành động điều chỉnh.
3	**Phân tích** **Test analysis**	"**Kiểm thử CÁI GÌ?**" – phân tích cơ sở kiểm thử, xác định điều kiện kiểm thử.
4	**Thiết kế** **Test design**	"**Kiểm thử NHƯ THẾ NÀO?**" – thiết kế test case, xác định dữ liệu cần thiết.
5	**Triển khai** **Test implementation**	Chuẩn bị testware: test script, test data, môi trường, sắp thứ tự thực thi.
6	**Thực thi** **Test execution**	Chạy test, so sánh kết quả thực tế với mong đợi, ghi nhận và báo cáo lỗi.
7	**Kết thúc** **Test completion**	Bàn giao testware, đóng báo cáo lỗi, tổng kết, rút kinh nghiệm.

Giám sát & kiểm soát diễn ra song song, xuyên suốt tất cả các hoạt động còn lại.

Sản phẩm kiểm thử (Testware) & Truy vết (Traceability)

TESTWARE – sản phẩm tạo ra trong kiểm thử

- Kế hoạch kiểm thử (Test plan)
- Lịch trình, sổ đăng ký rủi ro (Risk register)
- Điều kiện kiểm thử, bảng chuyển trạng thái
- Test case, test script, test data
- Kết quả thực thi (Test results / test log)
- Báo cáo lỗi (Defect report)
- Báo cáo tiến độ & báo cáo tổng kết kiểm thử

TRACEABILITY – truy vết hai chiều

- Liên kết **cơ sở kiểm thử ⇄ điều kiện kiểm thử ⇄ test case ⇄ kết quả ⇄ lỗi**
- **Lợi ích:**
  - Đánh giá **độ bao phủ yêu cầu (coverage)**
  - Phân tích **tác động (impact analysis)** khi yêu cầu thay đổi
  - Chứng minh **tuân thủ (compliance)** với tiêu chuẩn ngành
  - Báo cáo tiến độ theo ngôn ngữ nghiệp vụ, dễ hiểu với khách hàng

THỜI LƯỢNG: ≈ 0,75 TIẾT

1.8

Con người trong kiểm thử phần mềm

- Vai trò các bên liên quan
- Tâm lý kiểm thử
- Kỹ năng của tester
- Tính độc lập

Vai trò các bên liên quan trong hoạt động kiểm thử

Vai trò	Nhiệm vụ chính liên quan đến kiểm thử
Tester (Người kiểm thử)	Phân tích yêu cầu, thiết kế & thực thi test case, ghi nhận và báo cáo lỗi, kiểm thử lại sau khi sửa.
Developer (Lập trình viên)	Viết và chạy **unit test**, gỡ lỗi (debug), sửa defect, tham gia review mã nguồn.
Business Analyst (BA)	Làm rõ yêu cầu nghiệp vụ, cung cấp **cơ sở kiểm thử**, xác nhận tiêu chí chấp nhận.
Product Owner / Khách hàng	Xác định độ ưu tiên, thực hiện **kiểm thử chấp nhận (UAT)**, quyết định phát hành.
Test Manager / QA Manager	Lập **chiến lược & kế hoạch kiểm thử**, phân bổ nguồn lực, theo dõi tiến độ, báo cáo cho bên liên quan.

Whole-team approach (Agile): chất lượng là trách nhiệm của **cả nhóm**, không phải riêng tester.

Tâm lý kiểm thử (Tester Psychology)

TƯ DUY LẬP TRÌNH VIÊN (Developer mindset)

- Mục tiêu: **xây dựng** và hoàn thiện sản phẩm
- Tư duy **khẳng định**: "chức năng của tôi chạy được"
- Thiên kiến xác nhận (**confirmation bias**): khó tự thấy lỗi của chính mình
- Tập trung vào **giải pháp kỹ thuật**

TƯ DUY NGƯỜI KIỂM THỬ (Tester mindset)

- Mục tiêu: **tìm ra lỗi tiềm ẩn** trước khi người dùng gặp
- Tư duy **phản biện (critical thinking)**, hoài nghi lành mạnh
- Chú ý chi tiết, quan tâm **luồng ngoại lệ (negative path)**
- Đặt câu hỏi "**Điều gì xảy ra nếu...?**"

Giao tiếp là chìa khoá: báo lỗi về **sản phẩm**, không phê phán **con người**. Dùng dữ liệu khách quan, mô tả rõ các bước tái hiện, tránh ngôn từ quy kết.

Các mức độ độc lập kiểm thử (Levels of Test Independence)

BƯỚC 1

Mức 0 – Tự kiểm thử

Tác giả tự kiểm thử sản phẩm của mình.
Rẻ, nhanh nhưng **thiên kiến cao nhất**.

BƯỚC 2

Mức 1 – Đồng nghiệp

Người khác trong cùng nhóm phát triển kiểm thử.
Hiểu ngữ cảnh tốt, thiên kiến giảm.

BƯỚC 3

Mức 2 – Nhóm kiểm thử riêng

Nhóm test độc lập trong tổ chức.
Khách quan cao, cần **phối hợp tốt**.

BƯỚC 4

Mức 3 – Bên ngoài tổ chức

Thuê ngoài / kiểm định độc lập.
Khách quan **cao nhất**, chi phí lớn, ít hiểu nghiệp vụ.

Độc lập càng cao ⇒ khả năng phát hiện lỗi càng khách quan, nhưng **nguy cơ mất kết nối** với nhóm phát triển càng lớn. Thực tế thường **kết hợp nhiều mức**.

Kỹ năng cần thiết của người kiểm thử

Kỹ năng phân tích

Đọc hiểu yêu cầu, bóc tách điều kiện kiểm thử, phát hiện chỗ mơ hồ / mâu thuẫn.

Tư duy logic & phản biện

Suy luận nhân quả, đặt giả thuyết về lỗi, thiết kế test có mục đích.

Kỹ năng giao tiếp

Trình bày lỗi rõ ràng, thuyết phục bằng bằng chứng, tránh xung đột với developer.

Cẩn thận & kiên nhẫn

Thực thi lặp lại chính xác, ghi nhận đầy đủ bằng chứng (log, ảnh chụp màn hình).

Kiến thức nghiệp vụ (domain)

Hiểu lĩnh vực (ngân hàng, y tế, giáo dục...) để nhận ra sai lệch nghiệp vụ.

Kỹ năng kỹ thuật

SQL, đọc log, API, cơ bản về lập trình – nền tảng để tiến tới kiểm thử tự động.

Một báo cáo lỗi tốt cần: **tiêu đề rõ – các bước tái hiện – kết quả mong đợi – kết quả thực tế – môi trường – mức độ nghiêm trọng – bằng chứng đính kèm**.

THỜI LƯỢNG: ≈ 1 TIẾT

1.9

Bài tập vận dụng và câu hỏi ôn tập

- 4 bài tập
- 5 câu hỏi ôn tập

Phân tích lỗi phần mềm

- Một hệ thống bán hàng có yêu cầu:
  - **"Giảm giá 15% cho đơn hàng có giá trị từ 2.000.000 đồng trở lên."**
- Đoạn mã được lập trình như sau:
  - `if (total > 1000000)`
  - `  discount = total * 0.15;`
- **Yêu cầu:**
  - a) Xác định **Error**, **Defect** và **Failure** trong tình huống trên.
  - b) Đề xuất cách khắc phục (sửa mã nguồn).
  - c) Thiết kế các giá trị kiểm thử để phát hiện lỗi này.

Xây dựng quy trình kiểm thử

- Giả sử nhóm của bạn phát triển một **website đăng ký học phần** cho sinh viên.
- Hãy xây dựng quy trình kiểm thử gồm các bước sau, trình bày ngắn gọn công việc cần thực hiện ở mỗi bước:
  - ① Lập kế hoạch kiểm thử (Test Planning)
  - ② Phân tích và thiết kế kiểm thử (Test Analysis & Design)
  - ③ Triển khai và thực hiện kiểm thử (Test Implementation & Execution)
  - ④ Đánh giá và kết thúc kiểm thử (Test Evaluation & Closure)
- **Gợi ý:** với mỗi bước, hãy nêu rõ *đầu vào – công việc – sản phẩm đầu ra*.

Phân tích nguyên tắc kiểm thử

- Với mỗi nguyên tắc kiểm thử sau, hãy nêu **một ví dụ thực tế** minh hoạ việc áp dụng nguyên tắc đó trong quá trình phát triển phần mềm:
  - a) Không thể kiểm thử vét cạn (Exhaustive testing is impossible)
  - b) Kiểm thử nên thực hiện sớm (Early testing)
  - c) Lỗi thường tập trung theo cụm (Defects cluster together)
- **Yêu cầu mở rộng:** với mỗi ví dụ, chỉ rõ *hậu quả nếu KHÔNG áp dụng* nguyên tắc đó.

Xác định vai trò trong nhóm kiểm thử

- Một dự án phát triển phần mềm gồm các thành viên: **Product Owner, Business Analyst, Developer, Tester và Test Manager**.
- **Yêu cầu:**
  - a) Xác định vai trò và nhiệm vụ chính của từng thành viên trong hoạt động kiểm thử.
  - b) Giải thích vì sao sự phối hợp giữa các thành viên là cần thiết.
  - c) Nêu 2 hậu quả cụ thể khi giao tiếp trong nhóm không hiệu quả.

Bộ 10 bài tập tự luyện (Chương 1)

- **Bài 1.** Yêu cầu: "Cho phép rút tiền nếu số dư ≥ 50.000đ". Code: `if (balance > 50000) allowWithdraw();` → Chỉ ra Error, Defect, Failure.
- **Bài 2.** Một tester chạy 200 test case, tất cả đều Pass. Bạn ấy kết luận: "Phần mềm không còn lỗi". Kết luận này sai ở nguyên tắc nào? Vì sao?
- **Bài 3.** Ô "Tuổi" nhận giá trị 18–60. Hãy giải thích vì sao không thể kiểm thử vét cạn kiểu dữ liệu `int` và đề xuất tập giá trị kiểm thử tối thiểu.
- **Bài 4.** Bộ hồi quy 300 test case chạy suốt 6 tháng không phát hiện lỗi mới, nhưng khách hàng vẫn báo lỗi. Nguyên tắc nào bị vi phạm? Cách khắc phục?
- **Bài 5.** Phân biệt **Verification** và **Validation**. Cho ví dụ một sản phẩm "đúng đặc tả nhưng sai nhu cầu".
- **Bài 6.** Sắp xếp các hoạt động sau theo đúng thứ tự quy trình kiểm thử ISTQB: *thực thi – phân tích – kết thúc – lập kế hoạch – thiết kế – triển khai – giám sát & kiểm soát*.
- **Bài 7.** Một lỗi được phát hiện ở giai đoạn UAT thay vì giai đoạn review yêu cầu. Ước lượng chênh lệch chi phí và nêu 2 biện pháp shift-left.
- **Bài 8.** Cho 4 mức độ độc lập kiểm thử. Chọn mức phù hợp cho: (a) phần mềm điều khiển máy xạ trị; (b) website giới thiệu quán cà phê. Giải thích.
- **Bài 9.** Viết một **báo cáo lỗi (defect report)** đầy đủ cho tình huống: nhấn nút "Thanh toán" hai lần liên tiếp thì hệ thống tạo 2 đơn hàng trùng.
- **Bài 10.** Hệ thống quản lý bệnh viện. Hãy liệt kê 8 đặc tính chất lượng ISO/IEC 25010 và với mỗi đặc tính nêu **một câu hỏi kiểm thử** cụ thể.

Câu hỏi ôn tập chương 1

1. Trình bày khái niệm kiểm thử phần mềm và phân biệt ba khái niệm **Error – Defect (Bug) – Failure**. Cho ví dụ minh hoạ.
2. Phân tích vai trò của kiểm thử phần mềm trong việc **nâng cao chất lượng** sản phẩm và **giảm chi phí** phát triển.
3. Trình bày **bảy nguyên tắc** cơ bản của kiểm thử phần mềm. Theo bạn nguyên tắc nào quan trọng nhất? Giải thích.
4. Mô tả các bước trong **quy trình kiểm thử phần mềm** và cho biết mục tiêu của từng bước.
5. Phân tích vai trò của **yếu tố con người** trong kiểm thử. Một tester cần những kỹ năng nào để làm việc hiệu quả?
6. Phân biệt **kiểm thử (testing)** và **gỡ lỗi (debugging)**. Nêu 4 mức độ độc lập kiểm thử và ưu/nhược điểm của từng mức.

Tóm tắt chương 1

- ✓ Kiểm thử phần mềm là quá trình **đánh giá và xác minh**, bao gồm cả hoạt động **tĩnh** và **động**, diễn ra **xuyên suốt vòng đời** phát triển.
- ✓ **Error** → **Defect** → **Failure**: sai sót của con người tạo ra khiếm khuyết trong sản phẩm, khiếm khuyết khi được thực thi mới gây thất bại.
- ✓ Chi phí sửa lỗi **tăng theo cấp số nhân** theo giai đoạn phát hiện ⇒ nguyên tắc **kiểm thử sớm (shift-left)** là quan trọng bậc nhất.
- ✓ Bảy nguyên tắc kiểm thử định hướng **tư duy**; sáu cách tiếp cận cung cấp **phương pháp triển khai** – cần phối hợp linh hoạt theo ngữ cảnh.
- ✓ Quy trình kiểm thử gồm 4 bước cơ bản (ISTQB chi tiết thành 7 hoạt động) và có **tính lặp**, không phải chuỗi tuyến tính một chiều.
- ✓ Chất lượng là trách nhiệm của **cả nhóm**; tester cần **tư duy phản biện** và **kỹ năng giao tiếp** để báo lỗi hiệu quả mà không gây xung đột.

Đánh giá kết quả học tập – Tổng quan về kiểm thử phần mềm

BÀI QUIZ · 30%

- **15 câu trắc nghiệm**, thời gian 15 phút
- Làm vào **đầu buổi chiều** ngày 1, 2, 3
- Mỗi câu **0,67 điểm** – làm tròn thang 10
- Chấm chéo tại lớp, công bố đáp án ngay
- Lấy **trung bình 3 bài quiz**

BÀI KIỂM TRA VIẾT 45 PHÚT · 70%

- **20 câu trắc nghiệm + 1 bài tập vận dụng**
- Làm vào **đầu ngày 4**, phủ cả 6 chương
- Trắc nghiệm **5 điểm** – bài tập **5 điểm**
- Mỗi chương góp khoảng **3–4 câu** trắc nghiệm
- Bài tập vận dụng chọn **1 trong 6 đề** của các chương

CÁCH TÍNH ĐIỂM QUÁ TRÌNH

Điểm quá trình = **0,7 × điểm bài kiểm tra viết 45 phút** + **0,3 × trung bình 3 bài quiz**

Ví dụ: bài viết 8,0 và ba quiz lần lượt 7,0 – 8,0 – 9,0 (trung bình 8,0) → điểm quá trình = 0,7 × 8,0 + 0,3 × 8,0 = **8,0**

Quiz 15 câu – Chương 1 (phần 1/3)

Câu 1. Lập trình viên hiểu sai yêu cầu và viết dấu > thay vì >=. Việc HIỂU SAI đó gọi là:
A. Defect B. Error C. Failure D. Root cause

Câu 2. Dấu > sai nằm trong mã nguồn được gọi là:
A. Error B. Failure C. Defect D. Risk

Câu 3. Thí sinh đúng 15,0 điểm bị hệ thống từ chối. Hiện tượng này là:
A. Error B. Defect C. Failure D. Bug

Câu 4. Phát biểu nào ĐÚNG?
A. Mọi defect đều gây failure B. Mọi failure đều do defect C. Có defect không gây failure D. Defect và failure là một

Câu 5. Chạy 200 test case đều Pass. Kết luận đúng là:
A. Phần mềm không còn lỗi B. Chưa phát hiện lỗi trong 200 tình huống đã thử C. Phần mềm đạt chất lượng D. Có thể phát hành ngay

Quiz 15 câu – Chương 1 (phần 2/3)

Câu 6. Một ô nhập số nguyên 32 bit có khoảng 4,29 tỷ giá trị. Điều này minh hoạ nguyên tắc:
A. Kiểm thử sớm B. Lỗi tập trung C. Không thể kiểm thử vét cạn D. Nghịch lý thuốc trừ sâu

Câu 7. Bộ 300 test chạy 6 tháng không tìm ra lỗi mới. Đây là biểu hiện của:
A. Phần mềm đã hoàn hảo B. Nghịch lý thuốc trừ sâu C. Lỗi tập trung D. Ngộ nhận không lỗi

Câu 8. Phần mềm không còn lỗi nhưng xuất PDF trong khi người dùng cần Excel. Đây là:
A. Lỗi tập trung B. Kiểm thử sớm C. Ngộ nhận không lỗi D. Không vét cạn

Câu 9. Hoạt động nào sau đây là GỠ LỖI, không phải kiểm thử?
A. Viết test case B. Đặt breakpoint dò nguyên nhân C. Chạy lại test sau khi fix D. Tạo báo cáo lỗi

Câu 10. Sửa một lỗi ở giai đoạn vận hành đắt hơn giai đoạn lấy yêu cầu khoảng:
A. 2 lần B. 10 lần C. 100 lần D. 1.000 lần

Quiz 15 câu – Chương 1 (phần 3/3)

Câu 11. Kiểm thử tĩnh là loại kiểm thử:
A. Có chạy chương trình B. Không chạy chương trình C. Chỉ dùng cho code D. Chỉ máy làm được

Câu 12. Ai chịu trách nhiệm về chất lượng phần mềm?
A. Chỉ tester B. Chỉ developer C. Chỉ quản lý dự án D. Cả nhóm dự án

Câu 13. Thiên kiến xác nhận (confirmation bias) giải thích vì sao:
A. Tester giỏi hơn dev B. Người khó tìm lỗi trong sản phẩm của chính mình C. Phải test vét cạn D. Nên tự động hoá

Câu 14. Mức độ độc lập kiểm thử CAO NHẤT là:
A. Tác giả tự kiểm thử B. Đồng nghiệp cùng nhóm C. Nhóm độc lập trong tổ chức D. Đơn vị kiểm định bên ngoài

Câu 15. Một defect report tốt BẮT BUỘC phải có:
A. Tên developer gây lỗi B. Các bước tái hiện và kết quả mong đợi C. Dòng code bị sai D. Phương án sửa lỗi

Ngân hàng câu trắc nghiệm cho bài kiểm tra 45 phút – Chương 1

Câu 1. Một hàm xử lý ngày 29/2 bị viết sai nhưng năm nay không phải năm nhuận. Tình trạng hiện tại của hệ thống là:
A. Có failure, không có defect B. Có defect, chưa có failure C. Không có defect D. Có cả defect và failure

Câu 2. Nhóm phát hiện 80% lỗi nằm ở hai mô-đun thanh toán và khuyến mãi. Hành động PHÙ HỢP NHẤT là:
A. Chia đều thời gian test cho mọi mô-đun B. Dồn thêm công sức test vào hai mô-đun đó C. Ngừng test hai mô-đun đó D. Viết lại toàn bộ hệ thống

Câu 3. Lỗi "sai chính tả tên công ty ở trang chủ" trước ngày ra mắt sản phẩm được xếp:
A. Nghiêm trọng cao – ưu tiên cao B. Nghiêm trọng cao – ưu tiên thấp C. Nghiêm trọng thấp – ưu tiên cao D. Nghiêm trọng thấp – ưu tiên thấp

Câu 4. Một dự án bỏ hoàn toàn hoạt động review tài liệu yêu cầu để tiết kiệm thời gian. Hệ quả nghiêm trọng nhất là:
A. Tester không có việc làm giai đoạn đầu B. Lỗi yêu cầu bị phát hiện rất muộn, chi phí sửa tăng vọt C. Không viết được test case D. Không đo được độ bao phủ

Phân tích tình huống lỗi và viết báo cáo lỗi

Tình huống. Quy định tuyển sinh: "Thí sinh có tổng điểm **từ 15,0 trở lên** thì đủ điều kiện xét tuyển."

Lập trình viên cài đặt đoạn mã sau:

```
if (diem > 15.0)  ketQua = "Đủ điều kiện";
else              ketQua = "Không đủ điều kiện";
```

Thí sinh Nguyễn Văn A có tổng điểm 15,0 nhưng hệ thống báo "Không đủ điều kiện".

Yêu cầu.

a) Chỉ rõ đâu là **Error**, **Defect**, **Failure** trong tình huống trên. (1,5 đ)

b) Nêu **nguyên nhân gốc** (root cause) và đề xuất biện pháp ngăn lỗi tương tự tái diễn trong tương lai. (1,0 đ)

c) Viết một **báo cáo lỗi (defect report)** hoàn chỉnh cho tình huống này, đủ các thành phần đã học. (2,0 đ)

d) Đề xuất **mức Severity và Priority** cho lỗi, có giải thích. (0,5 đ)